

GitHub Copilot

AI Credits

A Practical Guide for Developers

Real scenarios

Auto mode guide

Credit-saving tips

Before & after examples

Everything you need to know to use GitHub Copilot effectively after the June 2026 switch from per-seat billing to **token-based AI Credits** — with zero jargon and real-world examples from everyday developer tasks.

June 2026 · v1.0

01 What Changed and Why It Matters

Before June 2026, GitHub charged a flat monthly fee per developer seat. You could use Copilot as much or as little as you wanted — one developer chatting all day paid the same as someone who barely touched it. That model is gone.

Now Copilot uses **AI Credits** — a token-based system where you pay for what you actually consume, measured in the tokens sent to and received from the AI model. Think of it like a mobile data plan: the heavier your usage, the more you spend.

■ The Coffee Shop Analogy

Old model (PRU): Like a monthly coffee subscription — pay once, drink as much as you want.

New model (AI Credits): Like paying per cup. A quick espresso (short chat) costs pennies. A 4-hour work session with a barista who builds your entire order from scratch (agent mode on a big codebase) costs significantly more.

The key insight: not all Copilot interactions cost the same anymore.

The One Rule That Changes Everything

There are two categories of Copilot interactions. Understanding which bucket your actions fall into is the most important thing in this guide:

Category	Examples	Cost	What to do
Always FREE	Inline code completions, Next Edit Suggestions (the grey ghost text as you type)	0 credits	Use these as much as possible — they never cost anything
Costs Credits	Chat window, Agent mode, Code Review, any time you press Enter to send a message to Copilot	Varies by model and context size	Be intentional — covered in detail throughout this guide

Bottom line: The grey ghost text that appears as you type is free. Anything in the chat panel or agent mode costs credits.

02 How Tokens and Credits Actually Work

Every time you send a message to Copilot, it packages up your message, your code context, and the conversation history into a bundle of **tokens** (roughly 1 token = 3–4 characters). This bundle goes to the AI model, the model generates a response (more tokens), and you're billed for both the input and the output.

What Gets Counted as Tokens

This is where many developers get surprised. It's not just your message that counts — it's everything Copilot includes in the bundle:

Token source	How big it gets	Your control over it
Your chat message	Small (100–500 tokens usually)	Full control — keep it focused
Files Copilot reads for context	Can be huge — a 500-line file is ~3,000 tokens	High control — use <code>#file:</code> to specify exactly what it reads
Conversation history	Grows with every turn — 10 turns can be 5,000+ tokens	Start fresh sessions to reset to zero
AI's response to you	Medium — detailed answers are more expensive	Ask for concise answers when detail isn't needed
Workspace index (when agent scans)	Very large — can be 20,000+ tokens on big codebases	Use <code>.copilotignore</code> to exclude irrelevant files

■ Why Agent Mode Gets Expensive Fast

When you ask agent mode to work on a task, it doesn't just answer once. It reasons through a plan, reads files, writes code, checks for errors, and often loops through these steps multiple times — each loop is a separate model call with the full context bundle attached.

A 10-step agent session on a large codebase can easily consume 80,000–200,000 tokens total, even if your original question was just a few sentences. The practices in this guide help you keep those numbers down.

03 Auto Mode — Our Organisation's Default

Our organisation uses **Auto mode** as the standard setting for all Copilot interactions. This is the recommended approach and the smartest default for managing credit costs. Here's why it matters and how to get the most from it.

What Auto Mode Actually Does

When you select Auto in the Copilot model picker, Copilot doesn't just pick a random model — it runs two systems simultaneously. One monitors real-time model availability and health. The other evaluates your task across dimensions like reasoning complexity, code generation difficulty, bug diagnosis depth, and tool orchestration needs. It then routes your request to the model that can solve it most efficiently at the lowest cost.

Auto mode benefit	What it means for you
10% credit discount	All paid plan users get a 10% reduction on every interaction when using Auto. A message that would cost 10 credits manually costs only 9 credits on Auto.
Task-aware routing	A quick question gets routed to Haiku 4.5 (cheap). A complex multi-file refactor gets routed to Sonnet 4.6. Auto matches cost to complexity automatically.
No frontier model surprises	Auto only selects from cost-efficient models (Haiku 4.5, Sonnet 4.6, GPT-5.4, GPT-5.3-Codex). It never routes to Opus or GPT-5.5 — the most expensive models.
Rate limit protection	If one model is under load, Auto instantly reroutes to an equivalent model so your workflow isn't interrupted. No manual model switching needed.
Full transparency	Hover over any Copilot response to see exactly which model was used. You always know what ran and can switch to a specific model any time.

✓ How to Confirm You're on Auto Mode

In VS Code: Look at the model picker dropdown at the top of the Copilot Chat panel. It should show "Auto" — not a specific model name like "Claude Sonnet" or "GPT-5.4".

In JetBrains IDEs: The model selector appears in the Copilot tool window. Select "Auto".

In GitHub.com (web): The model dropdown is visible above the chat input. Select "Auto".

If you see a specific model name, click the dropdown and select Auto to switch.

When to Override Auto and Pick a Specific Model

Auto mode handles the vast majority of tasks well. There are specific situations where overriding it with a manual model choice is justified — but these should be exceptions, not the default:

Situation	Override to	Credits per message	Why
Bulk unit test generation, boilerplate, simple Q&A;	Claude Haiku 4.5	~0.5–2 credits per message	Force cheapest model when you know the task is simple and volume is high
Deep architecture review, cross-repo migration, complex agent sessions	Claude Sonnet 4.6	~3–8 credits per message	Auto may pick Haiku for cost; override to Sonnet when quality matters more
Hardest reasoning — novel algorithm design, unfamiliar codebase deep-dive	GPT-5.4	~8–20 credits per message	Use sparingly; switch back to Auto after the specific task is done

■ Important: Switching Models Mid-Session Increases Cost

Research by GitHub shows that switching models mid-session increases costs without proportional quality improvement. If you override Auto for a task, finish that task, close the session, and start a fresh one when returning to Auto.

Auto routes along cache boundaries to avoid extra cache-related costs — manual mid-session switches lose this optimisation.

Full Model Reference — What Each Model Costs

Understanding what each model costs helps you judge when Auto's selection is correct and when you might want to override. Costs below are per typical developer interaction — not per token — to give you a practical sense:

Model	Typical quick chat (e.g. explain a function)	Typical agent session (e.g. refactor a class)	Typical large task (e.g. multi-file feature)	Auto eligible?
Claude Haiku 4.5	~0.5–1 credit	~3–8 credits	~10–25 credits	Yes
Claude Sonnet 4.6	~2–4 credits	~10–25 credits	~30–80 credits	Yes
GPT-5.4	~3–6 credits	~15–40 credits	~50–120 credits	Yes
GPT-5.3-Codex	~2–4 credits	~10–30 credits	~30–90 credits	Yes
GPT-5.5	~8–15 credits	~40–100 credits	~100–300 credits	No — manual only
Claude Opus 4.6	~20–40 credits	~100–250 credits	~300–800 credits	No — manual only

Credit estimates assume a medium-sized enterprise codebase (~40K LOC). Actual cost scales with how much code Copilot reads and how long its response is. Auto mode with 10% discount applies to all four 'Yes' rows above.

04 Real Scenarios — Where Credits Drain and How to Stop It

These are the most common ways developers unknowingly burn through credits. Each scenario shows exactly what went wrong and a concrete fix.

Scenario 1	The Vague Agent Prompt	
What happened	Dev opens agent mode and types: "Refactor the payment module to be cleaner." Agent scans the entire payments folder (8 files, ~2,400 lines), plans changes, asks clarifying questions, rewrites 3 files, hits an error, retries — all in one session.	
The credit problem	The word 'cleaner' forces the agent to make dozens of judgment calls. Each judgment call is another reasoning loop, another 15,000-token context bundle. The agent also scanned 6 files it didn't need to touch, adding ~18,000 tokens of irrelevant input on every single step.	
The fix	Be specific. Instead try: "Refactor #file:src/payments/PaymentService.ts to use constructor injection instead of property injection. Do not modify any other files." Agent reads one file, makes one targeted change, done.	
Credits saved: ~48 credits saved (from ~60 down to ~12)		Cost per run: ~60 credits (~\$0.60)

Scenario 2	The Marathon Chat Session	
What happened	Dev uses one Copilot Chat session all morning. Starts with "help me understand this auth flow", then uses the same session for "now write unit tests for it", then "refactor the login controller", then "explain this error". 15 turns total.	
The credit problem	Every message in a chat session re-sends the entire conversation history as input tokens. By turn 15, each new message carries 8,000+ tokens of old chat just as overhead — even if it's completely irrelevant to the current question. You're paying to resend your morning's entire conversation on every single turn.	
The fix	Treat each distinct task as its own session. Press 'New Chat' (the + icon in the Copilot panel) between tasks. Takes 2 seconds and resets your token overhead to near zero for each new task.	
Credits saved: ~30 credits saved — same output, 4 focused sessions instead of 1 long one		Cost per run: ~45 credits (~\$0.45)

Scenario 3	Asking Copilot to Read the Whole Codebase	
------------	---	--

What happened	Dev is building a new feature and types in agent mode: "Look at the existing code and write a new UserNotificationService following our patterns." Agent uses #codebase to scan the whole repo for "patterns" — reads 40+ files.
The credit problem	The phrase 'look at the existing code' with no specific reference triggers #codebase semantic search, which injects the top-matching chunks from dozens of files into every agent reasoning step. Most of those files are irrelevant — the agent just doesn't know that without you telling it.
The fix	Point to exactly what you want it to learn from: "Write a new UserNotificationService following the pattern in #file:src/services/EmailService.ts and #file:src/services/SmsService.ts. Use the interface from #file:src/interfaces/INotificationService.ts." Three specific files instead of 40+.
Credits saved: ~65 credits saved — from ~80 down to ~15 Cost per run: ~80 credits (~\$0.80)	

Scenario 4	Using Agent Mode for a 5-Line Change
What happened	Dev notices a function missing a null check. Opens the Copilot Chat sidebar, types: "Add null validation to the processOrder function in OrderService", and runs it as an agent task.
The credit problem	Opening a full agent task injects your entire workspace configuration, tool schemas, and file index into the context — even for a change that takes a human developer 30 seconds. The overhead far exceeds the actual task size.
The fix	Use inline chat (Ctrl+I in VS Code) instead. Select the function, press Ctrl+I, type 'Add null check at top'. Copilot only sees the selected code — no workspace scan, no tool schemas, no overhead. Same result at a fraction of the cost.
Credits saved: ~6–7 credits saved — from ~8 down to ~0.5–1 Cost per run: ~8 credits (~\$0.08)	

Scenario 5	The Legacy Migration Session Gone Wrong
What happened	Dev is migrating a Java Spring MVC service to Spring Boot 3. Types in agent: "Migrate all the services in src/main/java/com/company/services/ to Spring Boot 3." Agent works for 20 minutes across 12 service classes before hitting a cascading error.
The credit problem	Batch migration at this scale forces the agent to keep track of all 12 service classes simultaneously, building a massive context window. When it hits an error on service #8, it often retries the entire plan — consuming all those tokens again. One bad run can drain a third of your monthly credit allocation.

The fix

Migrate one service at a time. Each migration is its own fresh agent session: "Migrate `#file:src/main/java/com/company/services/PaymentService.java` to Spring Boot 3 constructor injection. Target: Spring Boot 3.2. Constraints: do not modify the controller layer, preserve all method signatures." Commit and run tests before starting the next service.

Credits saved: ~160 credits saved — same 12 services, systematic approach

Cost per run: ~200+ credits (~\$2.00+)

05 Maximise the Free Tools First

Inline completions and Next Edit Suggestions will never cost you a credit. Many developers underuse them because they seem passive, but with the right habits they can handle 60–70% of your daily coding without a single credit spent.

Writing Comments Before Code

Copilot's completions are significantly better when you give them a description to work from. Writing a detailed comment above a function tells Copilot exactly what you want — and in many cases it will generate the entire function body as a free completion:

```
// Comment-driven completions — 0 credits
// Fetches paginated user records from the database.
// Accepts: page (int), pageSize (int), optional filters object
// Returns: Promise<{ data: User[], total: number, hasNext: boolean }>
// Throws: DatabaseException if connection fails
async function fetchUsers(page: int, pageSize: int, filters?: UserFilters) {
// ← Copilot will complete the entire body here for FREE
}
```

The more specific your comment, the better the completion. Function signature, parameter descriptions, return type, and edge cases — all free input that Copilot uses without needing a chat turn.

Next Edit Suggestions — Your Free Refactoring Assistant

When you make a change to code, Next Edit Suggestions (enabled in VS Code settings) automatically predicts and suggests the next logical edit — like updating a parameter name everywhere it appears, or completing a pattern you started. Accept with Tab. It costs nothing and can chain through a refactoring sequence automatically.

✓ Turn These Settings On — They're Free and Save Credits

In VS Code settings, search "Copilot" and enable:

"Next Edit Suggestions: Enabled" — predicts your next code change automatically.

"Inline Suggest: Enable" — ensures completions appear as you type.

"Copilot: Enable for all languages" — so you get completions in every file type.

These settings do not consume any credits and often handle the small changes that would otherwise require a chat message.

06 Controlling What Copilot Reads — The .copilotignore File

Every file Copilot reads gets counted as input tokens. On a large enterprise codebase, a significant portion of your repository is files that Copilot will never need to read — build outputs, lock files, generated code, test fixtures. The .copilotignore file tells Copilot to skip these entirely.

Create a file called .copilotignore at the root of your repository (same syntax as .gitignore). Here's a ready-to-use template:

```
# .copilotignore
# .copilotignore – paste this at your repo root
# Files listed here are NEVER included in Copilot's context
# Build outputs and compiled files
dist/
build/
out/
*.min.js
*.bundle.js
target/ # Maven/Gradle output
# Package managers (these files are huge and useless to Copilot)
node_modules/
vendor/
package-lock.json
yarn.lock
pnpm-lock.yaml
Gemfile.lock
# Auto-generated code
src/migrations/ # Database migration files
**/*.generated.ts
**/__generated__ / # GraphQL codegen output
**/*.pb.go # Protobuf generated files
# Test data and snapshots
**/__fixtures__ /
**/testdata/
**/*.snap # Jest snapshots
# Reports and coverage
coverage/
.nyc_output/
```

✓ Real Impact of .copilotignore

A developer at a typical enterprise with 80,000 LOC in their repo tested this: before .copilotignore, agent mode read ~45,000 tokens of irrelevant files (node_modules references, lock files, generated DTOs) on every step.

After adding .copilotignore, the same tasks consumed ~12,000 tokens. That's a 73% reduction in input tokens — just from a 20-line config file.

07 Agent Mode — How to Use It Without Burning Credits

Agent mode is the most powerful — and most credit-intensive — way to use Copilot. Auto mode still applies its 10% discount and smart routing inside agent sessions, but because agent mode makes multiple model calls per task, the total still adds up fast. These practices keep agent sessions lean without sacrificing quality.

The #file: Reference — Your Most Important Habit

When you type a message in agent mode without specifying files, Copilot uses semantic search to guess what files are relevant and injects them all. When you use #file: to specify exactly which files matter, Copilot reads only those. This single habit is responsible for the biggest credit savings in agent mode.

```
# The #file: habit — single biggest credit-saver in agent mode
# BEFORE — Copilot guesses what's relevant (expensive)
"Write unit tests for the payment service"
# Copilot reads: PaymentService.ts, OrderService.ts, CartService.ts,
# UserService.ts, payment.types.ts, order.types.ts, and 8 more files
# Total context: ~35,000 tokens

# AFTER — you tell Copilot exactly what to read (cheap)
"Write unit tests for #file:src/payments/PaymentService.ts
following the pattern in #file:src/payments/PaymentService.test.ts
Mock the dependency at #file:src/payments/PaymentRepository.ts"
# Copilot reads: 3 files only
# Total context: ~4,200 tokens (88% reduction)
```

One Task Per Session

This is the second most impactful habit. Each message in an active session re-sends the entire conversation history. A session that starts with 'explain this code' and then pivots to 'now refactor it' and then to 'write tests for it' carries the full weight of every previous message on each new request.

Approach	Sessions	Approximate credits	Saving
One long session for 3 tasks	1 (15 turns)	~45 credits	—
Three separate sessions for 3 tasks	3 (5 turns each)	~18 credits	60% less

How to start a new session: Click the + (New Chat) icon in the Copilot panel sidebar. There is no memory between sessions — Copilot starts completely fresh, which is exactly what you want.

Always Review the Agent's Plan Before It Runs

When you start an agent task, Copilot shows you a plan — a list of files it intends to modify and steps it will take. Read this before clicking Continue. If the plan looks wrong (wrong files, wrong approach), cancel now. A single misdirected agent run on a complex task can consume 50–150 credits before you notice it's gone off track.

■ Three Questions to Ask Before Clicking Continue

1. Are the files in the plan correct? If it's listing files you didn't intend to change, cancel and rewrite the prompt with explicit #file: references.
2. Does the approach make sense? If the plan describes something fundamentally different from what you asked, the model misunderstood — cancel and clarify.
3. Is the scope appropriate? If it's proposing changes across 8 files when you expected changes in 2, that's a red flag. Narrow the prompt first.

Use Inline Chat (Ctrl+I) for Small Changes

Not everything needs a full agent session. For changes to a single function or a small block of code, the inline chat (Ctrl+I in VS Code) is dramatically cheaper because it only reads the code you've selected — nothing else.

Task size	Recommended approach	Why
Single function change	Select code → Ctrl+I → describe change	Only selected code is read — minimal tokens
Single file refactor	Ctrl+I on whole file, or Chat with #file:	Contained context, no workspace scan
Multiple file change	Agent mode with explicit #file: refs	Agent needed but control context carefully
Whole module / subsystem	Agent mode, one file/class at a time	Batch per session to limit context accumulation

08 Quick Reference — The 10 Rules

If you remember nothing else from this guide, these 10 rules will keep your credit usage under control.

1	Ghost text is free — use it aggressively	The grey inline completion is free. Use it for quick fixes and detailed comments and docstrings.
2	Always use Auto mode — it's cheaper and smarter	Auto gives you a 10% discount on credits compared to manual prompting. It never runs out of credits. Turn on Auto when you have credits left.
3	New task = new chat session	Click + (New Chat) button in the sidebar. Fresh session = fresh credits.
4	Use #file: — never let Copilot guess	Always tell agent mode which file you're working on. #codebase can read the whole repo, but #file: is more precise.
5	Use Ctrl+I for small changes	For anything touching a single file, use Ctrl+I. It reads the whole file and makes small changes. It's faster than agent mode.
6	Review the plan before you click Continue	Always read the agent's plan before clicking Continue. Each step costs credits. If you don't like the plan, click Stop.
7	Create a .copilotignore file	Add build outputs, logs, and other files to .copilotignore. It cuts input tokens by 50%.
8	Be specific in your prompts	Vague prompts ('make a function') cost more credits than specific ones ('make a function that takes a string and returns a boolean'). Specific prompts lead to fewer steps and less token usage.

9	Migrate one class at a time	For legacy migration and test after each o small.
10	Check your usage weekly	Visit GitHub Settings most credits. If you're

09 How Far Do Your Credits Actually Go?

In our organisation, base subscribers get **1,900 credits per month** and enterprise users get **3,900 credits per month**. Here's what a realistic working month looks like with good habits versus without them — using the base 1,900 credit allocation as the reference.

Scenario: Typical Month for a Backend Developer

Activity	Without good habits	With this guide's tips	Saving
Daily coding (completions)	0 credits (free)	0 credits (free)	—
Quick bug fixes and Q&A; (20x/month)	~80 credits (frontier model, long sessions)	~14 credits (Auto mode + short sessions)	~66 credits
Unit test generation (10 sessions)	~120 credits (whole-file agent scans)	~22 credits (Auto + #file: refs)	~98 credits
Feature development (3 features)	~300 credits (broad agent sessions)	~80 credits (Auto, scoped sessions)	~220 credits
Code review assistance (8 PRs)	~160 credits	~72 credits (Auto + inline chat small PRs)	~88 credits
Architecture / design discussions	~100 credits (frontier model by default)	~36 credits (Auto, new session each)	~64 credits
TOTAL	~760 credits (40% of 1,900 base allocation)	~224 credits (1,676 credits to spare)	~536 credits (70% reduction)

The 'With this guide's tips' column assumes Auto mode is active (10% discount applied), sessions are kept focused and short, and #file: references are used in agent mode. Actual consumption varies with codebase size and task complexity.

10 At a Glance — Do's and Don'ts

DO	DON'T
----	-------

- ✓ Use Tab completions and Next Edit Suggestions as your default — they're always free
- ✓ Keep Auto mode selected — it gives a 10% discount and never picks expensive models
- ✓ Start a new chat session for each separate task — history adds up as tokens
- ✓ Use `#file:path/to/file.ts` to tell Copilot exactly which files to read
- ✓ Use Ctrl+I inline chat for changes under ~50 lines — far cheaper than agent mode
- ✓ Create a `.copilotignore` to exclude build files, lock files, and generated code
- ✓ Review the agent plan before clicking Continue — cancel if scope looks wrong
- ✓ Break large migration jobs into one-service-at-a-time sessions
- ✓ Write detailed comments above functions to get better free completions
- ✓ Check your usage dashboard weekly — hover responses to see which model Auto picked

- ✗ Don't switch away from Auto mode unless you have a specific reason — it costs more
- ✗ Don't switch models mid-session — finish the task, then start fresh on a new model
- ✗ Don't run the same long chat session all morning across multiple tasks
- ✗ Don't let Copilot scan the whole codebase — use `#file:` instead of `#codebase`
- ✗ Don't manually pick GPT-5.5 or Opus for routine tasks — Auto never does this
- ✗ Don't run a migration agent across an entire module in one session
- ✗ Don't click Continue without reading the agent's proposed plan
- ✗ Don't leave build outputs and lock files accessible to Copilot
- ✗ Don't write vague prompts like 'make this better' — be specific about the change
- ✗ Don't expect a fallback model — that feature was removed in June 2026

The single most important takeaway: Our organisation uses Auto mode as the standard — and with good reason. It gives a 10% credit discount, routes tasks to the right model automatically, and never picks the expensive frontier models. Combined with the practices in this guide — short focused sessions, `#file:` references, and a good `.copilotignore` — you can do a full month of serious development work well within your 1,900 credit allocation.

Sources: GitHub Blog (April 27, 2026) · GitHub Docs — Copilot Billing & Models · GitHub Community Discussion #192948